CandNo: 262915

Advanced Natural Language Engineering (G5114): Assessed coursework

AI tool use statement

I acknowledge the use of ChatGPT (https://chatgpt.com/) to search for the relevant research papers, summarising articles, rephrasing my own paragraphs to reduce word count, and generating some Python code.

1 Introduction

Propaganda detection is a crucial task in natural language processing, aiming to identify not just whether propaganda is present, but also the specific techniques used. Accurately classifying propaganda techniques is important for supporting transparency in public discourse, combating misinformation, and developing responsible AI systems. However, this task is highly challenging due to overlapping techniques, subtle language patterns, and severe class imbalances between propaganda types. Moreover, span-level identification requires models to not only classify but also precisely localize propaganda spans, making the problem even more complex compared to traditional text classification.

2 Approaches and Related work

2.1 Propaganda Technique Classification (Task 1)

2.1.1 Approach 1 OvR with Logistic Regression

For the first approach of Task 1, a One-vs-Rest (OvR) strategy using static embeddings and Logistic Regression was adopted. Logistic Regression and SVM are both widely used for OvR due to their interpretability and effectiveness with high-dimensional data [6,7], but Logistic Regression was preferred as it directly provides probability estimates, avoiding the need for additional steps like Platt scaling required by SVM [8].

This approach mirrors early NLP work, where binary problems like sentiment classification were tackled using bag-of-words or embedding-based features such as Word2Vec and GloVe. While studies differ on the best static embeddings, research [9] suggests performance differences are often minor and task-dependent, motivating a sub-experiment comparing both embeddings.

OvR with static embeddings offers simplicity and effectiveness, particularly for imbalanced datasets [2], although the provided dataset does not show strong imbalance. However, OvR can suffer from issues like unclassified regions [1] and multiple labels per instance, which, while problematic in strict evaluation settings, might reflect real-world cases where spans contain multiple propaganda techniques. Since the evaluation requires a single label, the class with the highest decision function value is selected [1]. Traditional classifiers like OvR also struggle with modeling inter-class dependencies and capturing contextual subtleties, unlike transformer models such as BERT, which provide unified contextual representations.

2.1.2 Approach 2 ALBERT with CNN

Previous work in claim verification [3] and fine-grained sentiment analysis has shown that transformers excel at capturing subtle distinctions in language, which could be crucial for differentiating techniques like "exaggeration" and "loaded language." While BERT achieves strong results with minimal feature engineering, it is computationally intensive and less interpretable compared to traditional models. To address this, ALBERT [4], a lighter variant of BERT, is used in this project. ALBERT reduces parameter counts significantly while achieving better or comparable performance on several benchmarks [5].

In this approach, ALBERT is used solely for encoding—producing dynamic contextual embeddings for each token—without adding additional complex sequential models. Research [16] shows that adding LSTM, GRU, or BiLSTM on top of BERT often yields little gain while increasing

computational complexity. Specifically, [16] found that a CNN head outperformed other architectures like LSTM, BiLSTM, and GRU when used on top of BERT for text classification tasks. Therefore, a Text-CNN layer will be added after ALBERT, which helps to better capture local features in the text, mitigate overfitting, and improve overall model performance [5].

2.2 Span Identification and Technique Classification (Task 2)

2.2.1 Approach 1 BiLSTM-CRF

For the joint task, a BiLSTM-CRF model combining static embeddings (Word2Vec or GloVe) with handcrafted linguistic features is used. This approach, inspired by Lample et al. (2016) [10], has been widely adopted in Named Entity Recognition (NER) and other sequence labeling tasks. Feature-based BiLSTM-CRF models are particularly effective when domain knowledge is strong and datasets are limited, as they can explicitly incorporate syntactic and semantic cues. The BiLSTM processes the sentence in both directions, with each direction represented by a distinct network and its own parameters. These bi-directional representations help capture the contextual meaning of words, which is beneficial for tagging applications [10]. The CRF layer then predicts the most likely sequence of labels (BIO-style, including propaganda types) from the BiLSTM outputs.

To further assist the model in understanding context, additional linguistic and structural signals are extracted from the sentences. Unlike transformer models like BERT, which learn contextual features implicitly, traditional models rely on explicit input features for good performance. Incorporating features such as POS tags can highlight useful patterns; for instance, "Loaded language" often appears in adjectives or adverbs [2]. Research in [11] shows that POS tagging and NER labels significantly benefit sentence classification tasks, such as inferring malware actions. Additionally, [13] suggests that combining TF-IDF scores with word2vec embeddings can enhance classification performance, although care must be taken to normalize TF-IDF values to avoid overpowering dense embeddings. Therefore, in this approach, normalized TF-IDF scores, POS tags, and NER labels are included alongside static embeddings.

2.2.1 Approach 2 Fine-tuned ALBERT

As an alternative, a fine-tuned ALBERT model is used for token classification, predicting BIO-style labels that capture both span boundaries and technique types. BERT-based models have set state-of-the-art results for token classification across multiple domains [14], largely due to their ability to model long-range dependencies and polysemy through contextual embeddings. However, transformer models are often considered "black boxes" and may require significant data and computational resources for effective fine-tuning. They can also be prone to overfitting on small or noisy datasets if not carefully regularized. Research in [15] found that transformer-based models like BERT were highly effective for both sentence-level and fragment-level propaganda detection, and almost all top-ranking teams in SemEval 2020 Task 11 employed fine-tuned BERT models.

3 Methods

3.1 Evaluation metrics

To comprehensively evaluate model performance, three metrics were used: Overall Accuracy, Mean F1-score, and F1-score Standard Deviation. Overall Accuracy measures the proportion of correctly classified instances but can be misleading on imbalanced datasets. Mean F1-score, balancing precision and recall across all classes, provides a fairer assessment, especially with uneven class distributions. F1-score Standard Deviation highlights variability across classes; a low value indicates consistent performance, while a high value shows the model excels on some techniques but struggles on others—a key concern in propaganda detection. Together, these metrics provide a robust view of both overall and per-class performance. F1 score is also used for per-class evaluation as it harmonically balances precision and recall.

3.2 Propaganda Technique Classification (Task1)

By comparing OvR classifiers and the ALBERT-based classification (with a CNN head), this project aims to evaluate two fundamentally different approaches: one using separate binary classifiers for each technique (enabling potential multilabel assignment), and the other using a unified softmax classifier over contextual embeddings. Although the transformer-based approach is expected to outperform traditional classifiers overall, OvR's design allows for per-class hyperparameter tuning, which could benefit certain classes more than others.

3.2.1 Approach 1 OvR with Logistic Regression

For the OvR approach using static embeddings and Logistic Regression, the following design was used:

First, dataset analysis was performed to assess class imbalance and calculate average sentence and span lengths per class, identifying potential challenges. In preprocessing, non-propaganda examples were removed, and combined context-span inputs were extracted for training. Propaganda technique labels were converted to numerical classes.

Static embeddings were then applied. Since Word2Vec and GloVe produce word-level vectors, sentence embeddings were created by averaging all word vectors. To ensure fair comparison, both embeddings used 300 dimensions, with the largest vocabulary versions selected (GloVe: 2.2M vocab; Word2Vec: 3M vocab).

For the Logistic Regression classifier, the hyperparameters of interest include *C* (the regularization factor controlling generalizability), *solver* (the optimization algorithm), and *penalty* (the regularization type). All available solvers in scikit-learn (*lbfgs, liblinear, newton-cg, newton-cholesky, sag,* and *saga*) will be tested, as they are all suitable for binary classification tasks [17]. The penalty will be set to the default *l2* Ridge regularization, which is commonly more effective in text classification problems. Initially, the default *C=1.0* will be used, but further fine-tuning will explore different C values. Specifically, F1 scores will be recorded for each class while varying C in steps of 0.1 between 1.0 and 4.0, to find optimal regularization strengths both overall and per-class.

Performance was visualized by plotting per-class F1 scores across C values for both Word2Vec and GloVe, assessing which embedding or setting favored each technique. To allow per-class hyperparameter tuning, eight separate classifiers were manually trained, instead of using the scikit-learn OneVsRestClassifier.

3.2.2 Approach 2 Fine-tuned ALBERT

Following the same preprocessing steps as the first approach—where the span and context are combined and labels are encoded—the input data is prepared for the Transformer-based method. The process then proceeds as follows:

- Load a pre-trained ALBERT model.
- Tokenize the combined span and context texts using the ALBERT tokenizer.
- Convert the tokenized texts into embeddings via ALBERT's output.
- Feed the full hidden state embeddings into a CNN head for training.
- Optimize hyperparameters based on monitoring training and validation losses and accuracies over time.
- Evaluate the model's classification performance on a test set.

Since both validation and test data are needed, the original validation set is divided in half.

Unlike typical usage that relies on the [CLS] token, this approach uses full hidden states output from ALBERT, as recommended by [5]. This method enables the CNN layer to detect local patterns (e.g., bigrams, trigrams) and mimic traditional n-gram feature extraction.

Hyperparameter tuning in this project follows a staged approach. Initial values are selected based on findings from related work, with subsequent adjustments made based on validation performance. For the CNN-based ALBERT model, kernel sizes of 3, 4, and 5 are tested, as prior studies [5] did not specify a best-performing size. Learning rates are explored around a starting value of 1e-5, reflecting a balance between the smaller rate used in [5] (5e-6) and the larger rate recommended in [16] (3e-5). Dropout rates of 0.2 and 0.3 are compared, following settings used in both [5] and [16]. Performance metrics such as validation F1 score and loss are tracked throughout training to guide the selection of optimal hyperparameters. The goal is to find configurations that improve generalization without introducing overfitting, especially given the relatively small size of the dataset.

The layered architecture of the BERT-CNN classifier follows the design proposed in [5], and the implementation is based on the publicly available code from (https://github.com/ZeroxTM/BERT-CNN-Fine-Tuning-For-Hate-Speech-Detection-in-Online-Social-Media/blob/main/BertCnnFinal.ipynb).

3.3 Span Identification and Technique Classification (Task 2)

By comparing BiLSTM-CRF and fine-tuned ALBERT, this project aims to contrast a method requiring substantial feature engineering with a method that requires no manual feature extraction. The results may reveal whether explicit linguistic features still provide an advantage in modern NLP pipelines.

3.3.1 Approach 1 BiLSTM-CRF

To implement a BiLSTM-CRF model for span identification and technique classification, the first step is dataset preprocessing. This involves converting the <BOS> and <EOS> annotations into BIO tags that embed the propaganda technique label, allowing the model to learn both the span boundaries and the propaganda technique in a single pass. For example, the sentence "I want to get <BOS> our soldiers <EOS> out" would be converted to [O, O, O, O, B-flag_waving, I-flag_waving, O].

Each unique label—whether "O," "B-technique," or "I-technique"—is then mapped to a numerical ID, resulting in 19 unique labels. After labeling, feature extraction is performed, combining several types of token-level features:

- **Word2Vec embeddings**: 300-dimensional vectors, selected because they outperformed GloVe embeddings in Task 1.
- **TF-IDF features**: Representing the importance of a word within the corpus, normalized using the formula (tf-idf - idf_min) / (idf_max - idf_min) after finding the minimum and maximum IDF values from the corpus [21]. (1 feature)
- **POS tags**: One-hot encoded into 18 features representing different parts of speech.
- **NER labels**: One-hot encoded into 16 features representing named entities.

This results in a [sequence_length x 335] feature matrix for each instance.

Padding was added to both feature and label sequences to standardize input dimensions, with attention masks generated to distinguish real tokens from padding. Hyperparameters explored included the SGD learning rate, switching to Adam with a learning rate of 0.001, and experimenting with larger hidden state sizes. The model followed the design by Lample et al. (2016) [10], using a single-layer BiLSTM with 100-dimensional hidden states, SGD at 0.01, and gradient clipping at 5.0, which showed better performance than Adam [Kingma and Ba, 2014]. Variations based on other studies were also tested, such as a lower learning rate of 0.005 [18] and larger hidden states up to 256 combined with Adam [19], shown to improve sequence labeling tasks.

The BiLSTM-CRF model was evaluated using span-level precision, recall, and F1 score computed with the seqeval library. Only exact matches of span boundaries and labels were considered correct, as token-level accuracy can overestimate performance. Span-level evaluation offers a stricter and more realistic assessment for real-world propaganda detection [20].

The code of layered architecture for this model was inspired by the implementation on the publicly available code in (https://github.com/LiyuanLucasLiu/LM-LSTM-CRF/tree/master/model)

3.3.2 Approach 2 Fine-tuned ALBERT

For the fine-tuned ALBERT model, the same method for generating BIO labels and numerical label IDs as in the first approach was applied. Sentences were tokenized using the ALBERT tokenizer (AlbertTokenizerFast), aligning labels so the first subword retained the BIO label and subsequent subwords were masked with -100 to exclude them from loss computation. A pre-trained AlbertForTokenClassification model (albert-base-v2) was fine-tuned to predict BIO-style labels, capturing span boundaries and technique types. Padding and truncation were handled dynamically using Huggingface's DataCollatorForTokenClassification.

Initial hyperparameters followed recommendations from [22] and included a learning rate of 4e-5, with additional tests at 2e-5 and 1e-5 for improved stability (Devlin et al., 2019). A batch size of 8 was selected to balance GPU usage and gradient stability, with larger sizes also explored. Training was initially set for 15 epochs, with monitoring to prevent overfitting. A weight decay of 0.01 was applied, with higher values tested to address the predominance of 'O' labels in the BIO tag distribution.

As in Approach 1 for Task 2, evaluation focuses on span-level performance using the seqeval library, where only exact matches of span boundaries and technique labels are considered correct.

4 Results

4.1 Propaganda Technique Classification (Task1)

4.1.1 Approach 1 OvR with Logistic Regression

When testing different solvers for Logistic Regression on Word2Vec embeddings, no significant difference was observed, as all solvers achieved the same accuracy of **0.448**. Therefore, the default *lbfgs* solver was retained. Figure 1 shows that the best C value for Word2Vec embeddings was 2.8, yielding an accuracy of **0.473**, while Figure 2 shows that for GloVe embeddings, the optimal C value was 3.3.

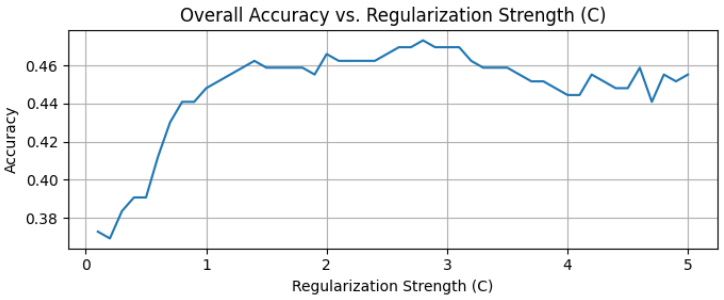


***Figure 1**. Word2Vec Optimal C exploration chart with the highest accuracy at C=2.8*

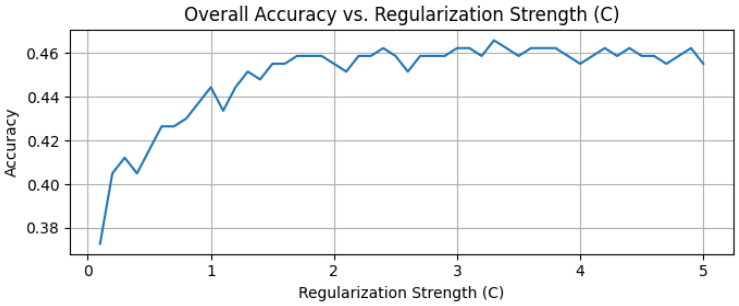


***Figure 2**. GloVe Optimal C exploration chart wit the highest accuracy at C=3.3*

Table 1 presents the best C parameters (1.0–4.0 range) for each binary classifier trained on Word2Vec or GloVe, based on F1 scores. The variation in optimal C values suggests that different classes respond differently to regularization, hinting at the potential for per-class optimization. However, applying the "best" C individually for each classifier did not improve overall performance. As shown in Table 2, performance either stagnated or slightly declined compared to using a single optimal C, likely because per-class optimization does not consider how classifiers interact during the argmax selection, which can explain the overall drop in metrics. This interaction between classifiers undermines per-class tuning benefits. Table 2 further shows that the best observed performance of **0.473** was achieved using Word2Vec embeddings with a uniform optimal C value of 2.8.

To compare per-class performance across embeddings, two line charts were generated and visualized in Figure 3. The results show some differences: for example, GloVe embeddings performed

better on the "repetition" technique, while Word2Vec performed noticeably better on "appeal to fear prejudice."

Table 1. *Best C value for each Binary classifier in OvR architecture trained on either Word2Vec or GloVe embeddings.*

Binary Classifier	Value of C	
	Word2Vec	GloVe
Appeal to fear prejudice	2.8	3.3
Causal oversimplification	1.9	1.9
Doubt	2.0	1.0
Exaggeration, minimisation	2.3	1.5
Flag waving	2.2	2.6
Loaded language	1.5	3.0
Name calling, labeling	1.8	1.9
Repetition	2.8	2.1

Table 2. *Result metrics representing the overall performance of OvR classifier using different C value strategies where Default C=1.0, Optimal C=2.8(Word2Vec)/3.3(GloVe), and Per-class optimised where C is picked to the class. Values in bold indicate best values in the column.*

Value of C (regularization)	Embedding	Accuracy	Mean F1	F1 Std Deviation
Default	Glove	0.444	0.427	0.119
	Word2Vec	0.448	0.424	0.118
Optimal	Glove	0.466	0.454	0.103
	Word2Vec	**0.473**	**0.455**	0.111
Per-class optimised	Glove	0.462	0.451	**0.099**
	Word2Vec	0.470	0.450	0.110

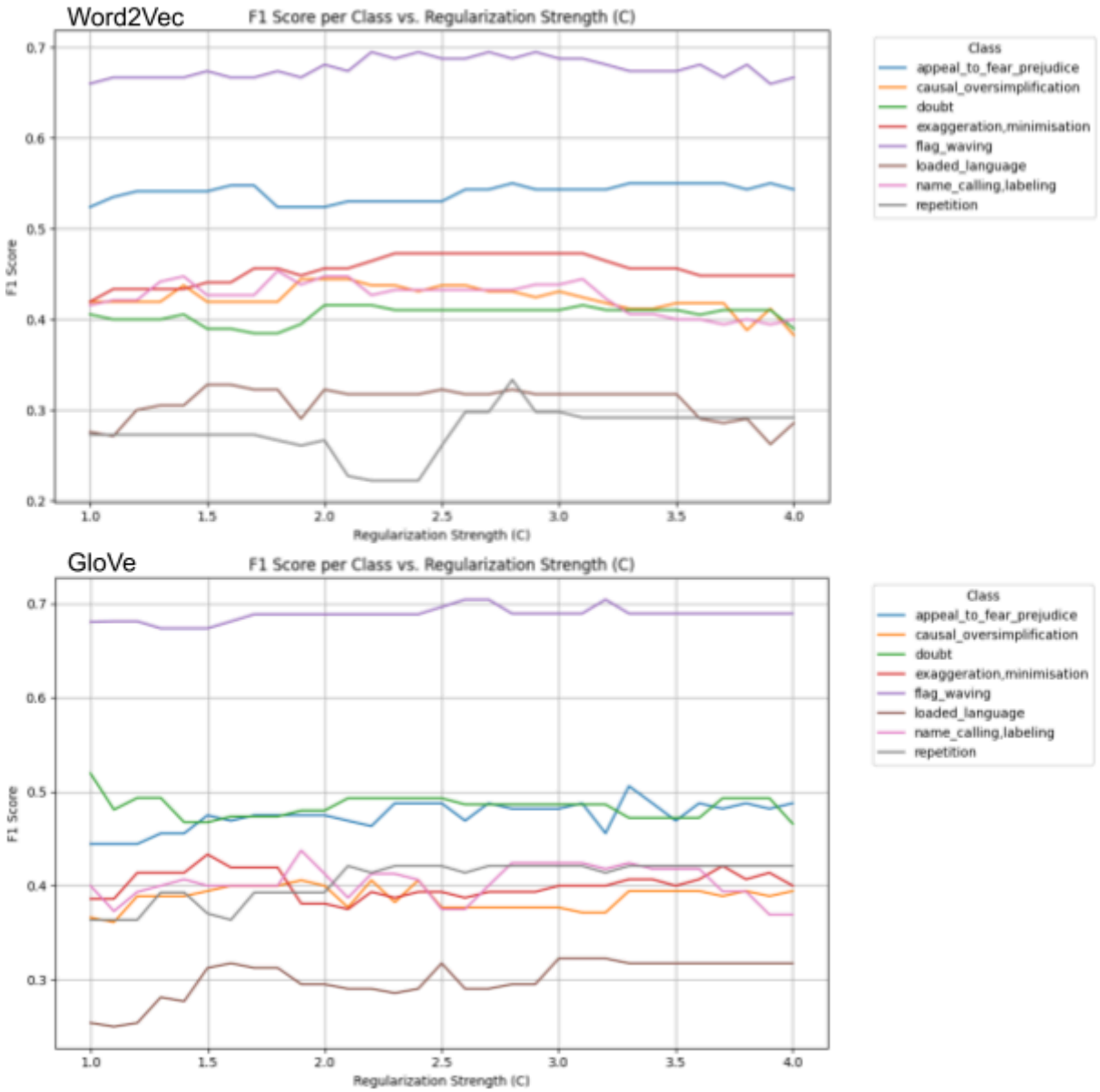


Figure 3. *F1-score over C factor for each binary classifier trained on Word2Vec (top chart) and GloVe (bottom chart) embeddings. The higher lines represent better classified techniques.*

4.1.2 Approach 2 Fine-tuned ALBERT

Table 3 summarizes the numerical results from hyperparameter experimentation, where parameters were explored sequentially: kernel size, number of epochs, dropout, and learning rate. The first experiment compared kernel sizes 3 and 4 using a learning rate of 1e-5 and dropout of 0.2 over 40 epochs. Increasing the kernel size nearly doubled the training time without improving performance (Table 3), so kernel size 5 was not tested. As shown in Figure 4, model (with kernel=3) performance plateaued after 20 epochs, with both training and validation losses increasing around epoch 35. Based on these observations, subsequent experiments were limited to 20 epochs.

When retraining with kernel size 3 and limiting to 20 epochs, the model achieved better accuracy and mean F1 score, although the F1 standard deviation slightly increased, suggesting higher variability across classes. Increasing the dropout rate to 0.4 further improved all performance metrics,

likely due to better generalization. The most effective learning rate was found to be 3e-5, achieving a classification accuracy of 0.6—comparable to top results from SemEval 2020 Technique Classification [15], where the highest accuracy was 63.63%. Training with a smaller learning rate of 5e-6 showed consistent improvement trends but did not reach convergence within the limited training time, as visualized in Figure 5.

Thus, the parameter combination of kernel size 3, dropout 0.4, and learning rate 3e-5 is suggested as the most optimal in terms of balancing training time and performance. However, the F1 standard deviation indicates that while overall performance is strong, classification performance varies across different classes.

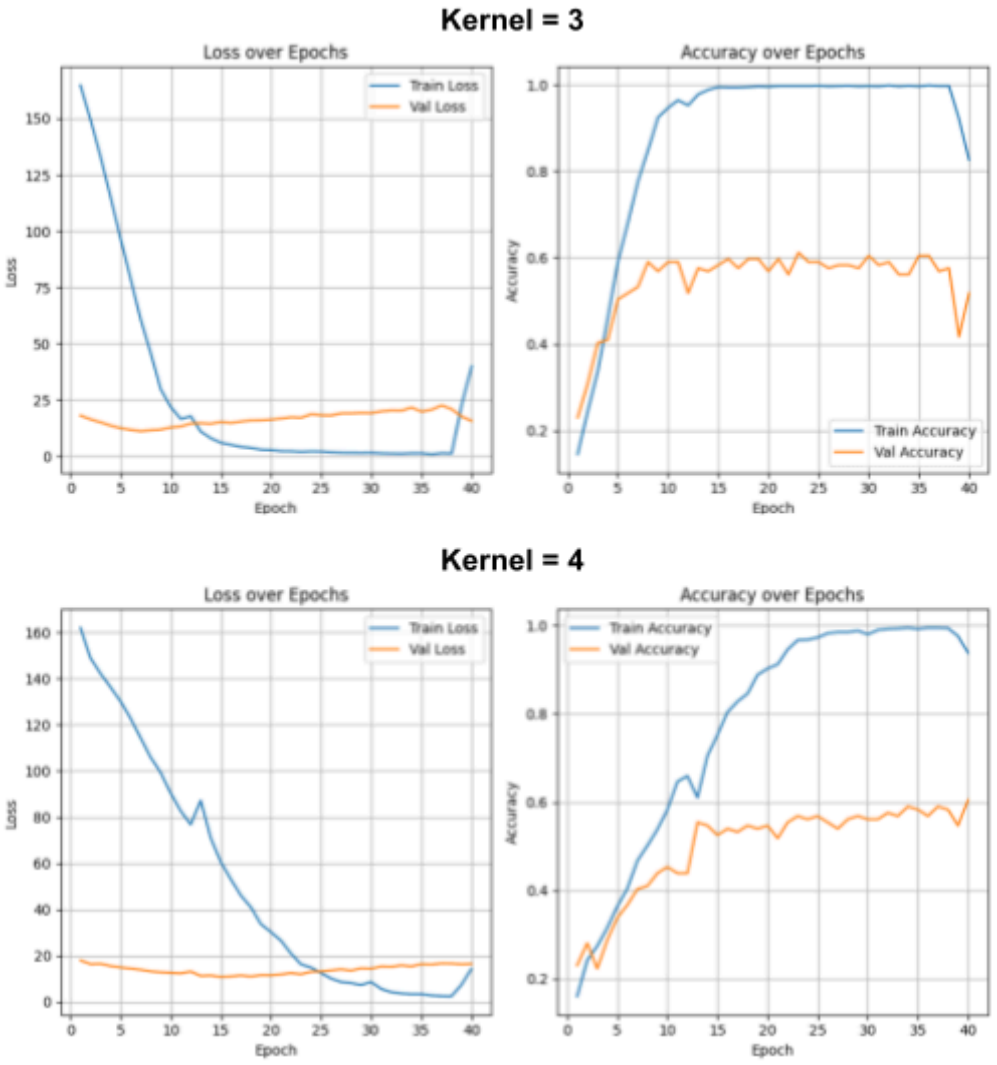


Figure 4. *Charts representing training and validation losses (on the left) and training and validation accuracies (on the right). Top two charts show model training using kernel size = 3, and two bottom charts show with kernel size = 4*

Table 3. *Performance metrics of AlbertCNN model using varied hyperparameters. Values in bold indicate best values in the column.*

Kernel Size	Epochs	Dropout	Learning Rate	Accuracy	Mean F1	F1 Std. Deviation
4	40	0.2	1e-5	0.421	0.403	0.147
3	40	0.2	1e-5	0.464	0.450	0.143
	20	0.2	1e-5	0.521	0.500	0.168
		0.3	1e-5	0.536	0.522	0.132
		0.5	1e-5	0.550	**0.589**	**0.069**
		0.4	1e-5	0.561	0.566	0.157
			3e-5	**0.600**	**0.589**	0.151
			5e-6	0.536	0.522	0.132

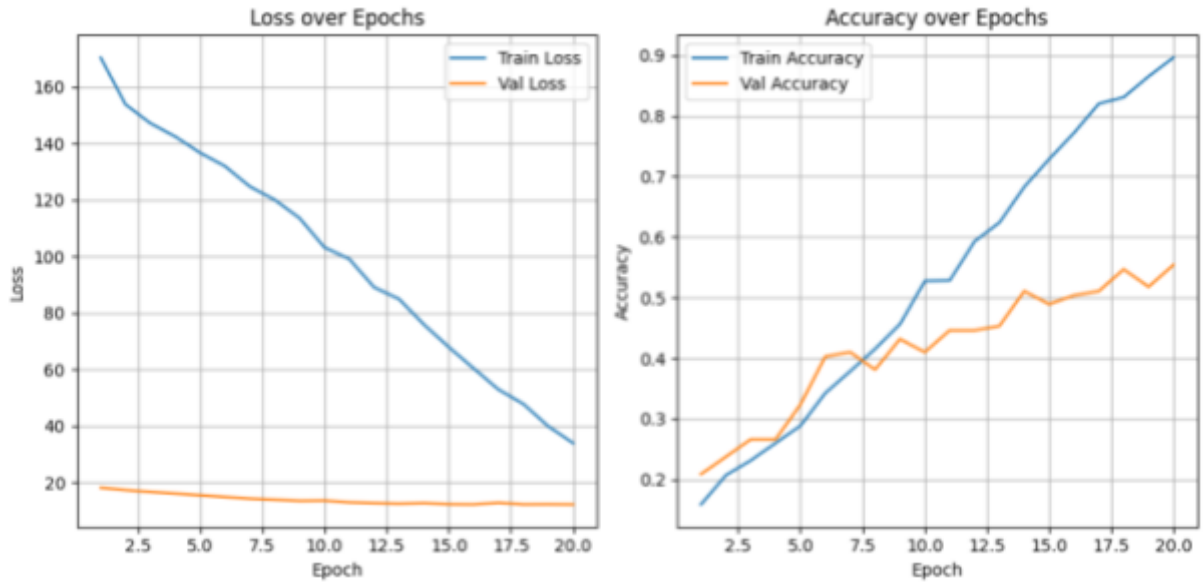


***Figure 5**. Charts representing training and validation losses (on the left) and training and validation accuracies (on the right) of the model trained with 5e-6 learning rate.*

4.1.3 Comparison of OvR Logistic Regression model and AlbertTextCNN model

As expected, the BERT based model performed better, however, the OvR model is much simpler to implement and much quicker to train. The baseline comparison of these two models presented in Table 4.

***Table 4**. Performance comparison of the model used in first approach - OvR Logistic Regression, and second approach - AlbertTextCNN.*

Model	Accuracy	Mean F1	F1 Std Deviation
OvR Logistic Regression	0.473	0.455	**0.111**
AlbertTextCNN	**0.600**	**0.589**	0.151

4.2 Span Identification and Technique Classification (Task 2)

4.2.1 Approach 1 BiLSTM-CRF

On the first attempt, the model's evaluation performance remained at 0.0 for all metrics across 10 epochs, despite a steadily decreasing training loss. Inspection revealed that the model predicted only "O" (outside) labels, exploiting the strong class imbalance where "O" covered 69% of the corpus.

To address this, a new dataset containing only propaganda instances was created, and training was conducted for 5 epochs at a reduced learning rate of 0.001 and other 5 epochs on the standard dataset with learning rate of 0.01, inspired by Curriculum Learning (Bengio et al., 2009). However, the model still predicted only "O" labels. An alternative strategy labeled entire non-propaganda sentences as a single span ("B-not_propaganda"/"I-not_propaganda"), reducing "O" tags, but this merely shifted the problem to predicting only "I-not_propaganda."

Following approaches used by SemEval-2020 participants (Da San Martino et al., 2020 [15]) which suggested resampling and class reweighting, I performed undersampling non-propaganda instances to 80% which tested without success as well as by switching to the Adam optimizer, and applying different gradient clipping (0.1-0.5). Finally, the CRF loss was modified to penalize excessive "O" predictions and the reduction method changed to "token_mean," which initially produced some class variation --- which looked rather random than predicted --- but quickly reverted to "O"-dominant outputs.

Overall, despite multiple strategies, the model failed to learn meaningful span detection, and realistic evaluation metrics were not achieved.

4.2.2 Approach 2 Fine-tuned ALBERT

Initial results were very poor, with an overall F1 score of 0.002. To address this, the same techniques as in approach 1 were applied: reformatting propaganda spans to cover the entire sentence, reducing "O" tags while increasing "not_propaganda" labels, followed by downsampling "not_propaganda" instances by 20% (from 1,191 to 941 examples). This significantly improved performance, raising the overall F1 score to 0.312, as it became easier for the model to identify non-propaganda spans. Training analysis (Figure 6) showed strong overfitting—training loss consistently decreased while validation loss increased—so the number of epochs was limited to 8 to maintain a better balance.

Different hyperparameter combinations, as described in the Methods Section, were tested but did not significantly improve performance. Most configurations resulted in the model primarily predicting non-propaganda spans correctly.

Substantial gains were achieved through a three-phase fine-tuning strategy:

- **Phase 1**: Learning rate = 4e-5, batch size = 8, epochs = 8, weight decay = 0.2.
  This helped generalize the model enough to start predicting some classes beyond

non-propaganda. However, extended training beyond 10 epochs caused the model to revert to only predicting "O" and "not_propaganda."

- **Phase 2**: Continuing fine-tuning with the same hyperparameters improved class balance.
- **Phase 3**: A final fine-tuning with a reduced learning rate (2e-5) and weight decay (0.1) achieved the best overall F1 score of 0.387.

Detailed results per phase are shown in Table 5, which highlights that Phase 3 yielded the best overall and per-class results (except for "Loaded language" and "Name calling," suggesting some forgetting occurred).

The model performed better on more frequent classes like "Not propaganda" and "Doubt," achieving F1 scores of 0.702 and 0.589, respectively. In contrast, it struggled with rarer techniques such as "Repetition," "Name calling, labeling," and "Loaded language." As noted in [23], some techniques are inherently more challenging: for example, "Flag-Waving" spans can vary significantly in form, and "Repetition" depends heavily on detecting recurring words within the broader article context. Therefore, span-level features alone may be insufficient, and future work should explore providing the model with full article context and word recurrence information [23].

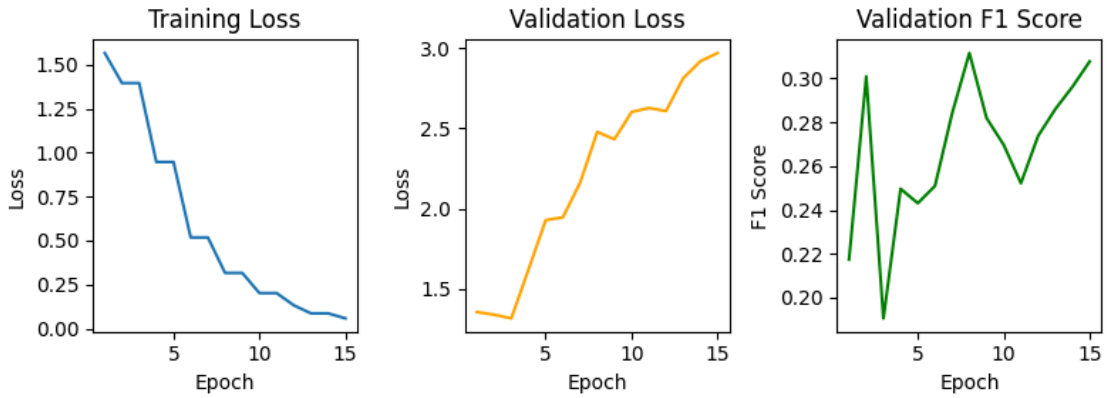


***Figure 6**. Training loss (left), validation loss (middle), validation f1-score (right) of the first phase of fine-tuning ALBERT for propaganda sequence classification during 15 epochs.*

Analysis showed a positive relationship between technique classification performance and tag occurrence frequency. Comparing the top-3 and bottom-3 techniques by F1 score (Table 5) and by tag frequency (Figure 7) indicates that higher-frequency techniques are generally classified more accurately. Figure 7 also highlights that while technique classes (excluding "not propaganda") are relatively balanced in count, their span lengths vary considerably.

The model also captured the general BIO tagging pattern, typically predicting a "B" tag followed by zero or more "I" tags, surrounded by "O" tags. This suggests it has learned basic structural patterns and could further improve with deeper contextual modeling.

***Table 5.** f1-scores of the ALBERT model during fine-tuning.*

Propaganda technique	f1-score at phase:

	Phase 1	Phase 2	Phase 3
Not propaganda	0.540	0.669	**0.702**
Doubt	0.000	0.040	**0.589**
Causal oversimplification	0.000	0.066	**0.215**
Appeal to fear prejudice	0.000	0.070	**0.149**
Exaggeration, minimisation	0.000	0.118	**0.133**
Flag waving	0.042	0.115	**0.130**
Loaded language	0.000	**0.063**	0.056
Repetition	0.000	0.000	**0.040**
Name calling, labeling	0.000	**0.087**	0.030
Micro Average	0.333	0.334	**0.387**
Macro Average	0.065	0.136	**0.168**

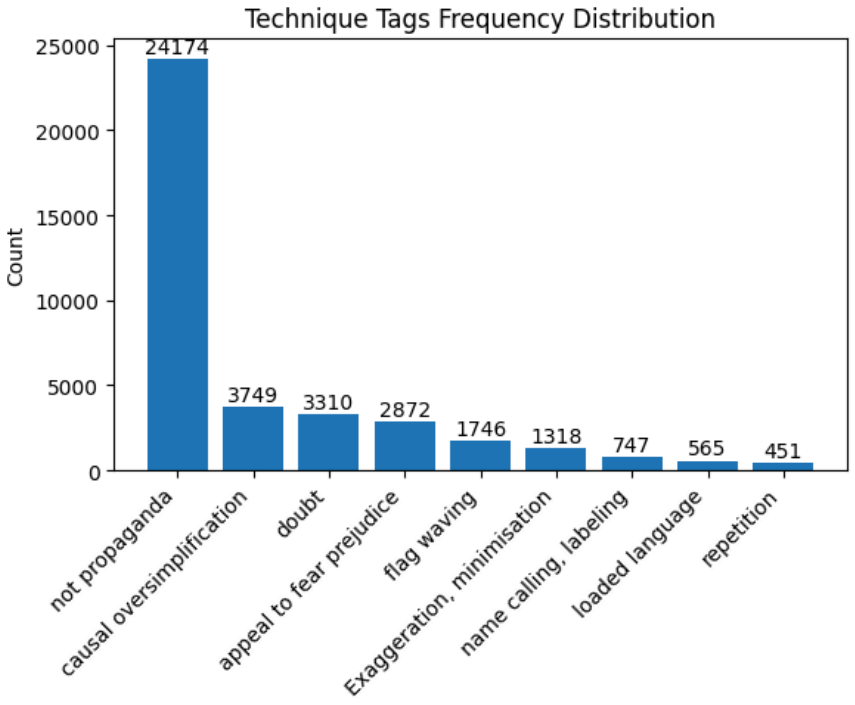


***Figure** 7. Frequency of BIO tags for each class in the corpus where I and B tags are summed together for better visualisation.*

5 Conclusion and Further work

This project investigated two tasks using both traditional and transformer-based methods. For Task 1, the OvR Logistic Regression approach with static embeddings provided a simple and interpretable baseline, while the ALBERT with CNN model achieved significantly better performance, particularly after hyperparameter tuning and dataset pre-processing.

For Task 2, span identification proved much more challenging. The BiLSTM-CRF model, even with engineered features, struggled severely due to extreme label imbalance and sequence complexity. Fine-tuned ALBERT showed greater potential, and after a three-phase fine-tuning strategy, achieved notable improvements with an overall F1 score of 0.387, however still not reaching the state-of-art performance for this task of 0.53 from SemEval 2020. Nevertheless, classification performance varied greatly across propaganda techniques, especially for rarer classes, highlighting the persistent challenges of class imbalance and limited span context.
Several promising avenues for future research emerge from these findings:

- **Task 1, Approach 1:** Given that Word2Vec embeddings worked better for some techniques and GloVe for others, future work could explore combining both embeddings dynamically, selecting the most appropriate embedding type per class to improve overall performance.

- **Task 2, Approach 1:** A potential extension would be to implement a class-weighted CRF loss, better handling label imbalance by penalizing errors on rare spans more heavily. As the standard torchcrf library does not support this natively, a custom CRF loss would be required.

  Research such as [12] suggests that BiLSTM-CRF models enhanced with character-level and word-level embeddings, combined with attention mechanisms, outperform standard BiLSTM-CRF on sequence labeling tasks. Incorporating these techniques could be explored to better capture complex span patterns.

References

[1] Ge, H., & Wang, L., 2014. Secondary Classification on One-against-Rest Multiclassification Structure. *Advanced Materials Research*, 1049-1050, pp. 1622 - 1625. https://doi.org/10.4028/WWW.SCIENTIFIC.NET/AMR.1049-1050.1622.

[2] Van de Kauter, M., Breesch, D. and Hoste, V. (2015). Fine-grained analysis of explicit and implicit sentiment in financial news articles. *Expert Systems with Applications*, 42(11), pp.4999–5010. doi:https://doi.org/10.1016/j.eswa.2015.02.007.

[3] Neema Kotonya and Francesca Toni. 2020. Explainable Automated Fact-Checking for Public Health Claims. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7740–7754, Online. Association for Computational Linguistics.

[4] Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma, and Radu Soricut. 2020. ALBERT: A lite BERT for self-supervised learning of language representations. In Proceedings of the 8th International Conference on Learning Representations (ICLR-2020).

[5] Zhu, X., Wang, J., & Zhang, X., 2021. YNU-HPCC at SemEval-2021 Task 6: Combining ALBERT and Text-CNN for Persuasion Detection in Texts and Images. , pp. 1045-1050. https://doi.org/10.18653/v1/2021.semeval-1.144.

[8] Caruana, R., & Niculescu-Mizil, A., 2005. An Empirical Comparison of Supervised Learning Algorithms Using Different Performance Metrics.

[9] Shi, T., & Liu, Z., 2014. Linking GloVe with word2vec. *ArXiv*, abs/1411.5595.

[10] Lample, G., Ballesteros, M., Subramanian, S., Kawakami, K. and Dyer, C. (2016). *Neural Architectures for Named Entity Recognition*.

[11] Chunping Ma, Huafei Zheng, Pengjun Xie, Chen Li, Linlin Li, and Luo Si. 2018. DM_NLP at SemEval-2018 Task 8: neural sequence labeling with linguistic features. In *Proceedings of the 12th International Workshop on Semantic Evaluation*, pages 707–711, New Orleans, Louisiana. Association for Computational Linguistics.

[12] Yao, L., Cheng, Y., & Li, C., 2021. Sequence labeling model based on hierarchical features and attention mechanism. *Journal of Physics: Conference Series*, 1883. https://doi.org/10.1088/1742-6596/1883/1/012020.

[13] Zhou, M., Liu, D., Zheng, Y., Zhu, Q., & Guo, P., 2020. A text sentiment classification model using double word embedding methods. *Multimedia Tools and Applications*, 81, pp. 18993 - 19012. https://doi.org/10.1007/s11042-020-09846-x.

[14] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. pages 4171–4186, Minneapolis, Minnesota. Association for Computational Linguistics.

[15] Giovanni Da San Martino, Alberto Barr´on-Cede˜no, Henning Wachsmuth, Rostislav Petrov, and Preslav Nakov. 2020. SemEval-2020 task 11: Detection of propaganda techniques in news articles.

[16] Kurniasih, A. and Manik, L.P. (2022). On the Role of Text Preprocessing in BERT Embedding-based DNNs for Classifying Informal Texts. *International Journal of Advanced Computer Science and Applications*, 13(6). doi:https://doi.org/10.14569/ijacsa.2022.01306109.

[17]https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[18] Ma, X. and Hovy, E. (2016). *End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF*.

[19] Reimers, N. and Gurevych, I. (2017). *Optimal Hyperparameters for Deep LSTM-Networks for Sequence Labeling Tasks*.

[20] Ibrahim, M. (2023). *A Guide to Unlocking the Power of Sequence Classification*. [online] W&B. Available at: https://wandb.ai/mostafaibrahim17/ml-articles/reports/A-Guide-to-Unlocking-the-Power-of-Sequence-Classification--VmlldzozNDI0NDE4.

[21] 4. The Effects of Feature Scaling: From Bag-of-Words to Tf-Idf (2015). *Feature Engineering for Machine Learning*. [online] O'Reilly | Safari. Available at: https://www.oreilly.com/library/view/feature-engineering-for/9781491953235/ch04.html.

[22] Kim, H. (2020). *Fine-Grained Propaganda Detection and Classification with BERT*. [online] Medium. Available at: https://medium.com/@jihwangk/fine-grained-propaganda-detection-and-classification-with-bert-dfad4acaa321 [Accessed 26 Apr. 2025].

[23] Altiti, O., Abdullah, M. and Ruba Obiedat (2020). JUST at SemEval-2020 Task 11: Detecting Propaganda Techniques Using BERT Pre-trained Model. *International Conference on Computational Linguistics*. doi:https://doi.org/10.18653/v1/2020.semeval-1.229.